Documentação Técnica — EcoTrack

Cidades ESG Inteligentes

Curso: Análise e Desenvolvimento de Sistemas **Turma:** [turma] **Integrantes:** Guilherme Barreto Ramos, Matheus Masques de Lima, Thaissa Kailaine Siqueira Oliveira

1. Descrição do Projeto

O EcoTrack é uma API REST desenvolvida em Java com Spring Boot, com o objetivo de gerenciar pontos de coleta e descarte de resíduos em cidades, incentivando práticas sustentáveis alinhadas aos princípios de ESG (Environmental, Social and Governance).

A aplicação expõe endpoints para cadastro, consulta, atualização e remoção de pontos de coleta, conectada a um banco de dados Oracle e implantada em nuvem com deploy automatizado.

2. Pipeline CI/CD

Ferramenta utilizada

GitHub Actions — integrado diretamente ao repositório GitHub, sem necessidade de servidor externo.

Arquivo de configuração

`.github/workflows/deploy.yml`

Etapas do pipeline

O pipeline é disparado automaticamente a cada `push` na branch `main` e é dividido em 4 etapas sequenciais:

Etapa 1 — Build

- Realiza o checkout do repositório
- Configura o JDK 21 (distribuição Temurin)
- Compila o código-fonte com `./mvnw clean compile`

Etapa 2 — Testes Automatizados

- Executa todos os testes da aplicação com `./mvnw test`
- Só avança para o deploy se os testes passarem

Etapa 3 — Deploy Staging

- Gera o pacote `.jar` com `./mvnw clean package -DskipTests`
- Realiza o deploy no ambiente de homologação do Azure App Service
- Utiliza a action oficial `azure/webapps-deploy@v3`
- Credenciais gerenciadas via GitHub Secrets

Etapa 4 — Deploy Produção

- Executada somente após o staging ser concluído com sucesso
- Publica o mesmo pacote `.jar` no ambiente de produção do Azure App Service

Secrets configurados no repositório GitHub

Secret	Descrição
`AZURE_WEBAPP_NAME_STAGING`	Nome do App Service de staging
`AZURE_WEBAPP_PUBLISH_PROFILE_STAGING`	Credencial de deploy do staging
`AZURE_WEBAPP_NAME_PROD`	Nome do App Service de produção
`AZURE_WEBAPP_PUBLISH_PROFILE_PROD`	Credencial de deploy da produção

3. Containerização com Docker

Dockerfile

```
FROM eclipse-temurin:21-jdk

WORKDIR /app
```

```
COPY . .

RUN chmod +x mvnw

RUN ./mvnw clean package -DskipTests

EXPOSE 8080

CMD ["java", "-jar", "target/EcoTrack-0.0.1-SNAPSHOT.jar"]
```

Estratégias adotadas

- Imagem base oficial `eclipse-temurin:21-jdk` para garantir compatibilidade com Java 21
- Build do JAR dentro do próprio container, sem necessidade de pré-compilar localmente
- Porta 8080 exposta, compatível com o Azure App Service

docker-compose.yml

```
version: '3.8'

services:
  ecotrack:
    build: .
    container_name: ecotrack-app
    ports:
      - "8080:8080"
    environment:
      - SPRING_DATASOURCE_URL=${SPRING_DATASOURCE_URL}
      - SPRING_DATASOURCE_USERNAME=${SPRING_DATASOURCE_USERNAME}
      - SPRING_DATASOURCE_PASSWORD=${SPRING_DATASOURCE_PASSWORD}
    volumes:
      - app-logs:/app/logs
    networks:
      - ecotrack-network
    restart: unless-stopped

volumes:
  app-logs:
```

```
networks:
  ecotrack-network:
    driver: bridge
```

Arquitetura Docker

- **Serviço:** `ecotrack` — a aplicação Spring Boot
- **Volume:** `app-logs` — persistência dos logs da aplicação
- **Rede:** `ecotrack-network` — rede isolada do tipo bridge
- **Variáveis de ambiente:** injetadas via arquivo `.env` (não versionado), com modelo em `.env.example`

Comandos utilizados

```
# Subir a aplicação localmente
docker compose up -d --build

# Acompanhar logs em tempo real
docker compose logs -f

# Parar a aplicação
docker compose down
```

4. Ambientes de Deploy

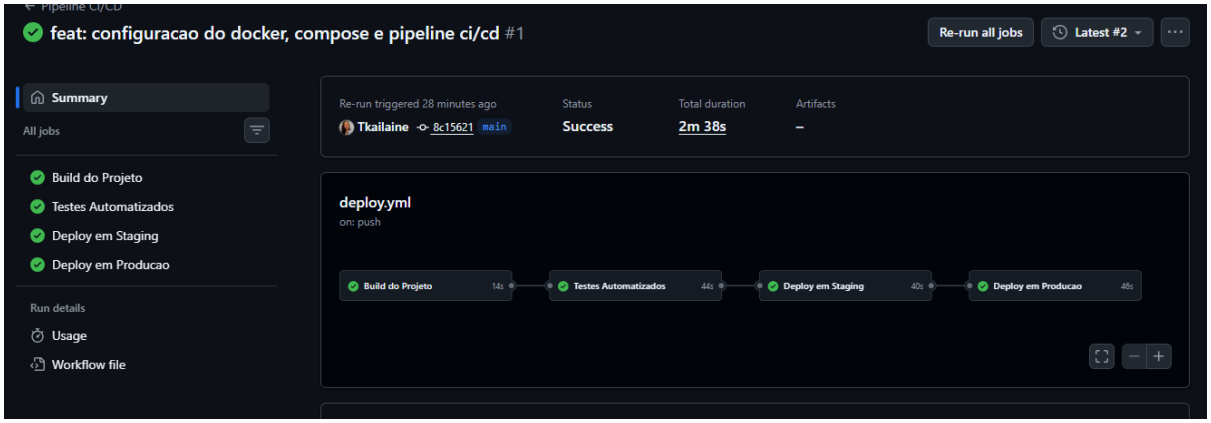
Ambiente	URL
Staging	https://ecotrack-staging-ebgve0ghemape4ct.southafricanorth-01.azurewebsites.net
Produção	https://ecotrack-prod-egc2csamgqabakak.southafricanorth-01.azurewebsites.net

Endpoint principal

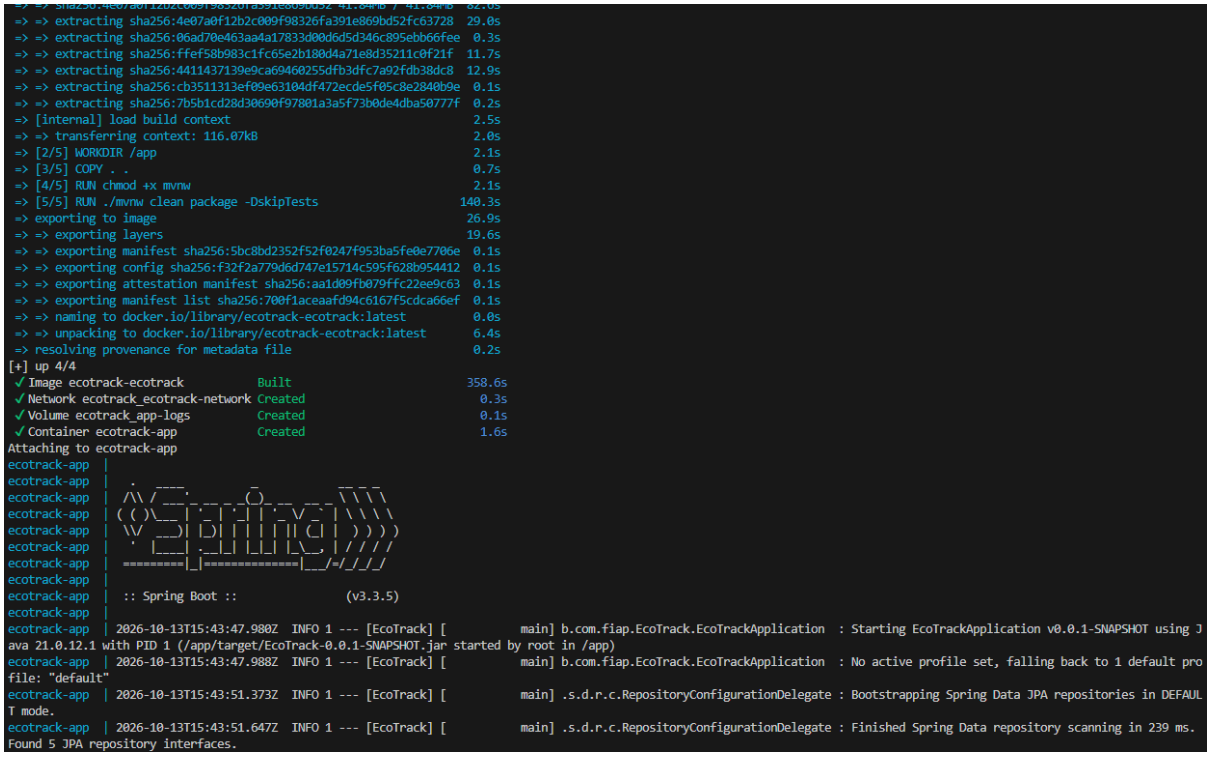
`GET /api/pontos-coleta` — retorna a lista de pontos de coleta cadastrados.

5. Prints de Evidência

5.1 Pipeline CI/CD rodando no GitHub Actions



5.2 Execução local via Docker



5.3 Ambiente de Staging funcionando

https://ecotrack-staging-ebgve0ghemape4ct.southafricanorth-01.azurewebsites.net/api/pontos-coleta

Estilos de formatação


```
[
  {
    "idPonto": 1,
    "local": "Centro",
    "capacidadMax": 500
  },
  {
    "idPonto": 2,
    "local": "Zona Sul",
    "capacidadMax": 600
  },
  {
    "idPonto": 3,
    "local": "Zona Norte",
    "capacidadMax": 400
  },
  {
    "idPonto": 4,
    "local": "Zona Leste",
    "capacidadMax": 350
  },
  {
    "idPonto": 5,
    "local": "Zona Oeste",
    "capacidadMax": 450
  }
]
```


5.4 Ambiente de Produção funcionando

https://ecotrack-prod-egc2csamgqabakak.southafricanorth-01.azurewebsites.net/api/pontos-coleta

Estilos de formatação


```
[
  {
    "idPonto": 1,
    "local": "Centro",
    "capacidadMax": 500
  },
  {
    "idPonto": 2,
    "local": "Zona Sul",
    "capacidadMax": 600
  },
  {
    "idPonto": 3,
    "local": "Zona Norte",
    "capacidadMax": 400
  },
  {
    "idPonto": 4,
    "local": "Zona Leste",
    "capacidadMax": 350
  },
  {
    "idPonto": 5,
    "local": "Zona Oeste",
    "capacidadMax": 450
  }
]
```

6. Tecnologias Utilizadas

Tecnologia	Função
Java 21	Linguagem principal
Spring Boot 3.3.5	Framework da aplicação
Oracle Database	Banco de dados relacional
Flyway	Migrations do banco de dados
Maven	Gerenciador de dependências e build
Docker	Containerização da aplicação
Docker Compose	Orquestração local dos serviços
GitHub Actions	Pipeline de CI/CD
Azure App Service	Hospedagem em nuvem (staging e produção)

7. Desafios Encontrados e Soluções

Desafio 1 — Cota do plano gratuito F1 no Azure A criação do App Service com o plano gratuito F1 falhou por ausência de cota disponível na região padrão. Solução: alterar a região para `South Africa North`, onde havia cota disponível.

Desafio 2 — Autenticação básica desabilitada no Azure O download do Publish Profile foi bloqueado porque a autenticação básica estava desativada por padrão. Solução: habilitar "SCM Basic Auth Publishing Credentials" nas configurações gerais do App Service.

Desafio 3 — Variáveis de ambiente não configuradas no Azure Após o deploy, a aplicação não conectava ao banco Oracle. Solução: configurar as variáveis `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` e `SPRING_DATASOURCE_PASSWORD` diretamente nas Application Settings do Azure App Service.

8. Checklist de Entrega

Item	Status
Projeto compactado em .ZIP com estrutura organizada	✅
Dockerfile funcional	✅
docker-compose.yml com volumes, redes e variáveis	✅
Pipeline com etapas de build, teste e deploy	✅
README.md com instruções e prints	✅
Documentação técnica com evidências (PDF)	✅
Deploy realizado nos ambientes staging e produção	✅